CM

CoreMotiom

Fluxograma completo do usuário e catálogo de casos de uso

Plataforma & marketplace esportivo de alta performance

Next.js 15 · React 19 · TypeScript · Supabase / PostgreSQL · Tailwind CSS v4

7	1	48	14
diagramas de fluxo	diagrama UML de casos de uso	casos de uso catalogados	lacunas e riscos mapeados

Gerado em 1 de outubro de 2026 a partir do código-fonte de BIRFeuFeu/-CoreMotion

7 fluxos de usuário · 1 diagrama UML de casos de uso · permissões por papel · máquina de estados · lacunas e riscos.

Sumário

A numeração abaixo segue a ordem de leitura do PDF. Cada diagrama ocupa uma página própria, com legenda de atores e de tipos de seta.

Seção	Página
1. Visão geral e mapa dos diagramas	3
1. Visão geral - do primeiro acesso às jornadas por papel	5
2. Autenticação, papéis e ciclo de sessão	6
3. Jornada de compra - descoberta, carrinho, pagamento e custódia	7
4. Jornada de venda - C2C, loja oficial e selo verificado	8
5. Engajamento - comunidade, treinadores e favoritos	9
6. Painel administrativo - 8 módulos executivos	10
7. Dados, serviços e persistência - onde cada ação é gravada	11
8. Diagrama de casos de uso	12
9. Catálogo completo de casos de uso	13
10. Papéis e permissões por tela	15
11. Máquina de estados do domínio	15
12. Lacunas e riscos encontrados no fluxo	16
13. Arquivos-chave por fluxo	17
14. Manutenção deste documento	18

1. Visão geral e mapa dos diagramas

Este documento reúne o mapa completo das jornadas de usuário da plataforma CoreMotiom e o catálogo de casos de uso, extraídos diretamente do código-fonte: telas renderizadas, modais globais, guardas de acesso, serviços, rotas de API e a migração SQL.

Rotas/telas renderizadas: 7 views: home, marketplace, stores, sell, coaches, community, admin

Modais globais: AuthModal · SupabaseConfigModal · CheckoutModal · CreateStoreModal · ProductDetailModal

Tabelas PostgreSQL: profiles · stores · products · orders · community_posts

Rotas de API: /api/products · /api/stores · /api/listings · /api/users · /api/auth/sync

Services de domínio: AuthService · ProductService · StoreService · OrderService · CommunityService

Como ler os diagramas

- Cada diagrama é uma seção numerada; os identificadores UC01 do diagrama de casos de uso correspondem ao catálogo da seção 9.
- Cores identificam o perfil do ator: cinza (visitante), verde (atleta), azul (lojista) e vermelho (admin master).
- As setas têm semântica: fluxo padrão, autenticação/sessão, sucesso, erro/bloqueio, ação administrativa e gravação de dados.
- Nós âmbar tracejados, marcados com (!), documentam lacunas ou comportamentos parciais no código atual.

Mapa dos diagramas

#	Diagrama	O que cobre
1	Visão geral - do primeiro acesso às jornadas por papel	Mapa macro de tudo que o app entrega hoje: bootstrap da SPA, detecção de sessão, papéis (visitante, atleta, lojista, admin), as seis jornadas de uso e a camada de persistência.
2	Autenticação, papéis e ciclo de sessão	AuthModal (4 modos), acesso rápido de demonstração, login por e-mail/senha, cadastro com tipo de conta, Google OAuth via popup + /auth/callback, guarda de admin e logout.
3	Jornada de compra - descoberta, carrinho, pagamento e custódia	Fluxo completo de compra: Home e busca global, filtros do marketplace, ficha do produto, carrinho, entrega, três meios de pagamento, criação do pedido e ciclo de status até a entrega.
4	Jornada de venda - C2C, loja oficial e selo verificado	Dois caminhos de oferta: atleta anunciando equipamento seminovo (C2C com custódia) e lojista abrindo/enviando loja para auditoria de CNPJ, com a homologação feita pelo admin.
5	Engajamento - comunidade, treinadores e favoritos	Fluxos sociais e de serviço: feed com 5 categorias, publicação, curtidas e comentários; agendamento de assessoria esportiva com CREF; e a lista de desejos.
6	Painel administrativo - 8 módulos executivos	Guarda de acesso por e-mail master, barra de ações rápidas e os módulos: métricas, auditoria de lojas, catálogo, usuários, pedidos/custódia, lojas e banco de dados.

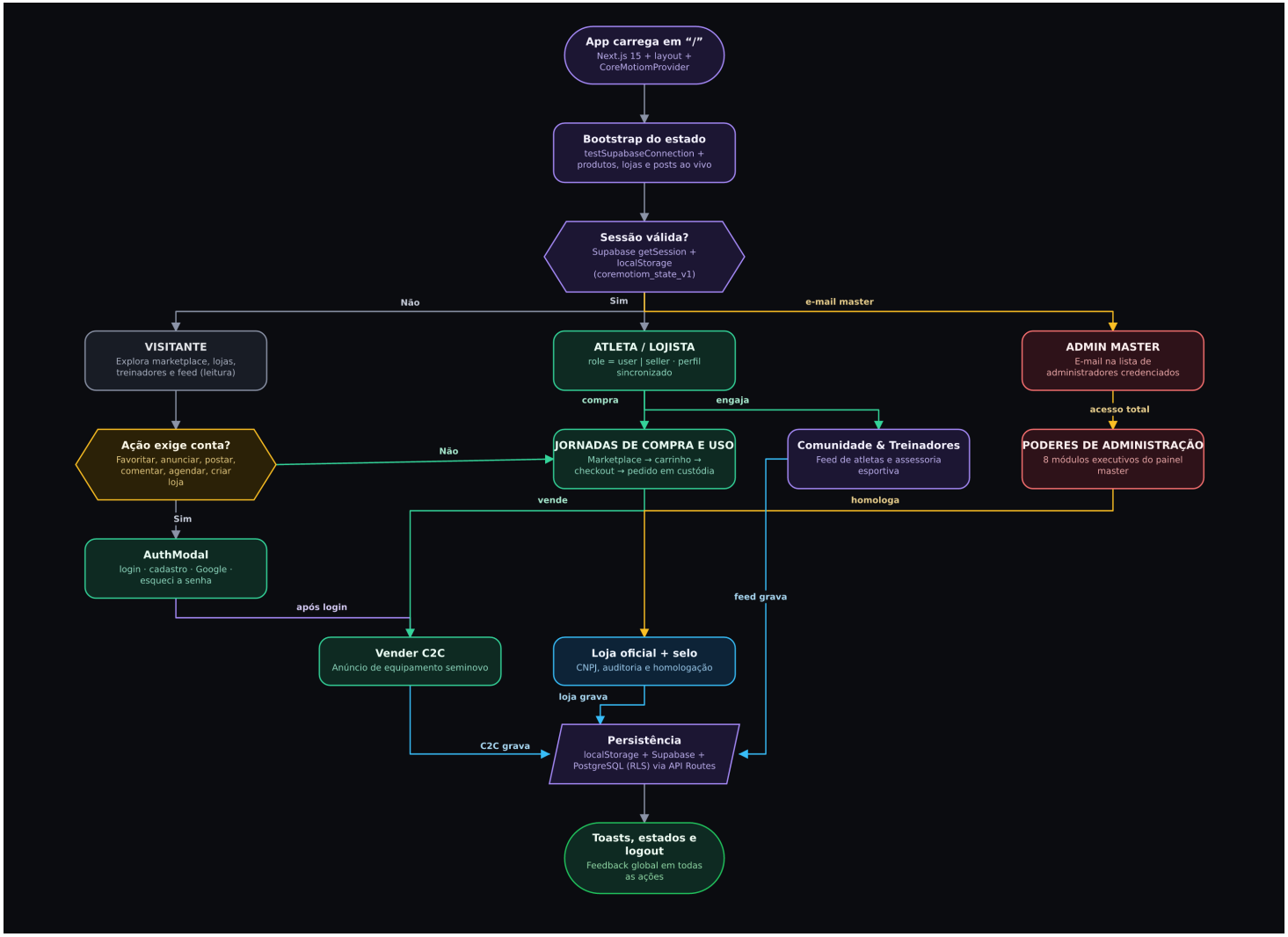
#	Diagrama	O que cobre
7	Dados, serviços e persistência - onde cada ação é gravada	Camadas do sistema: estado global no cliente, services que falam com o Supabase e com as API Routes, pool PostgreSQL com RLS, server actions validadas por zod (hoje sem uso) e o fallback offline.

Para leitura com zoom, os mesmos diagramas estão disponíveis em docs/fluxograma-usuario.html, gerado a partir deste mesmo núcleo de dados.

1. Visão geral - do primeiro acesso às jornadas por papel

Mapa macro de tudo que o app entrega hoje: bootstrap da SPA, detecção de sessão, papéis (visitante, atleta, lojista, admin), as seis jornadas de uso e a camada de persistência.

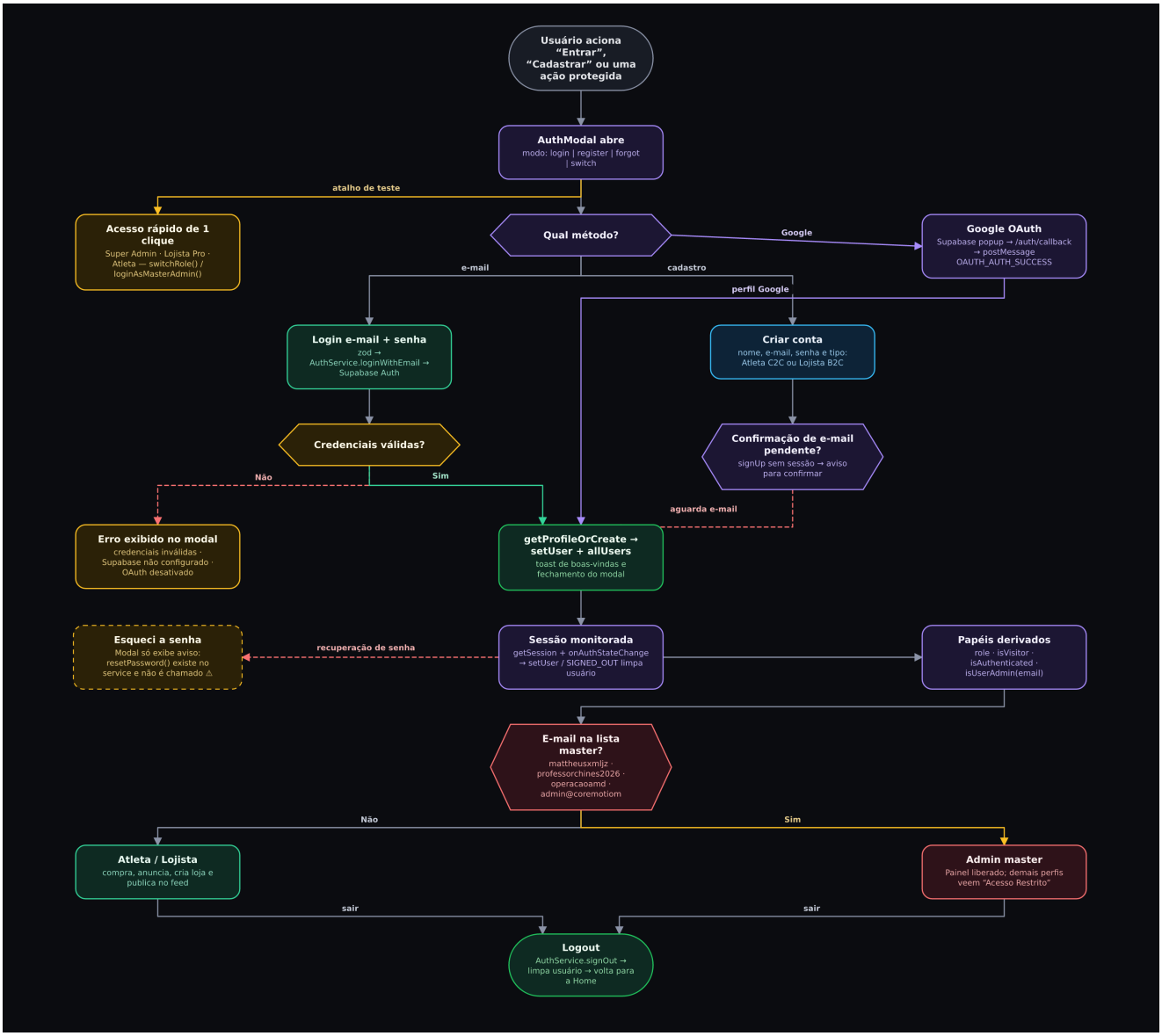
Legenda: Visitante (sem conta) · Atleta / Usuário (role: user) · Lojista (role: seller) · Super Admin (e-mails master) · Sistema / Dados



2. Autenticação, papéis e ciclo de sessão

AuthModal (4 modos), acesso rápido de demonstração, login por e-mail/senha, cadastro com tipo de conta, Google OAuth via popup + /auth/callback, guarda de admin e logout.

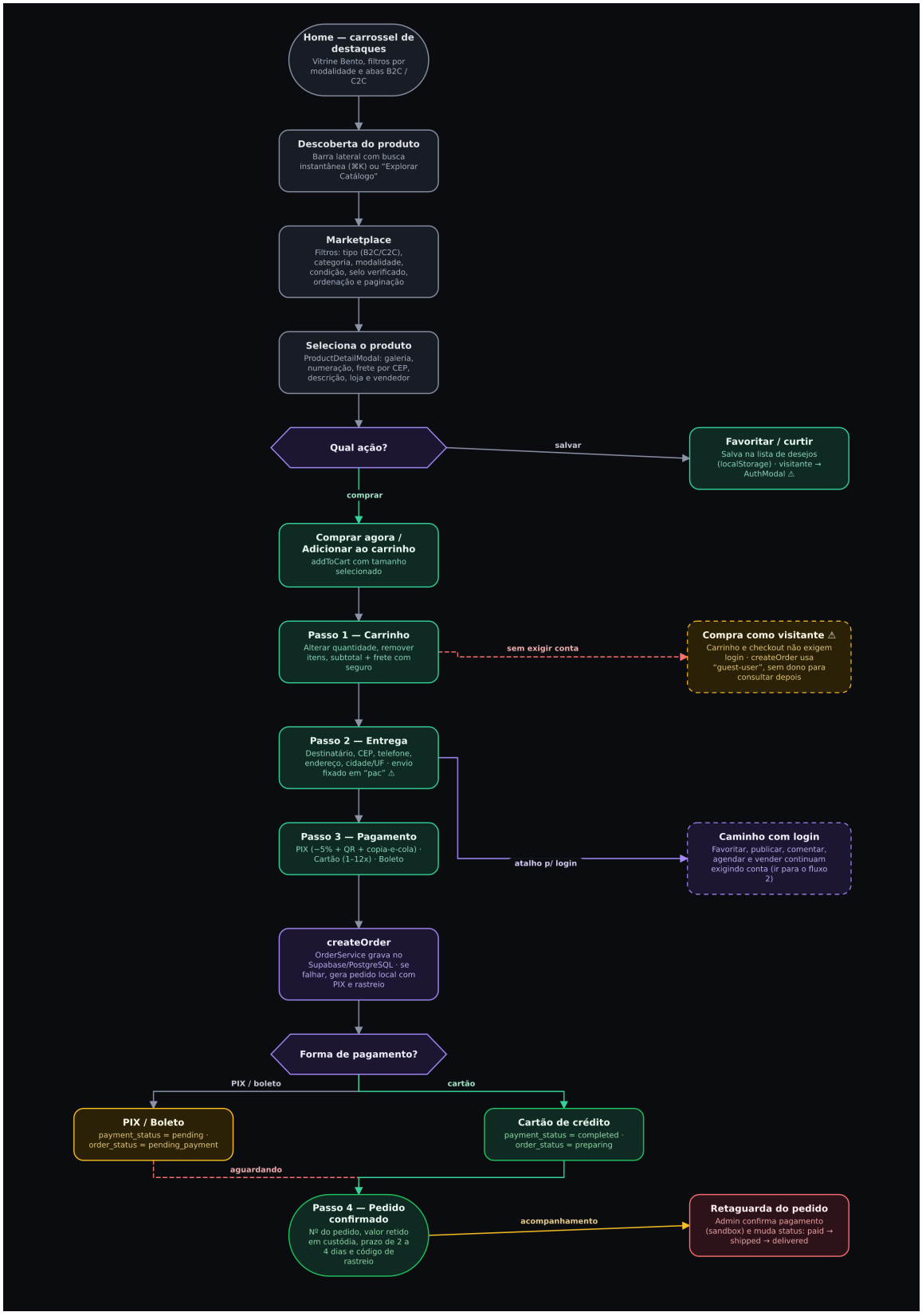
Legenda: Atleta / Usuário (role: user) · Lojista (role: seller) · Super Admin (e-mails master) · Alerta / Lacuna / Erro · Sistema / Dados · Confirmação / Sucesso



3. Jornada de compra - descoberta, carrinho, pagamento e custódia

Fluxo completo de compra: Home e busca global, filtros do marketplace, ficha do produto, carrinho, entrega, três meios de pagamento, criação do pedido e ciclo de status até a entrega.

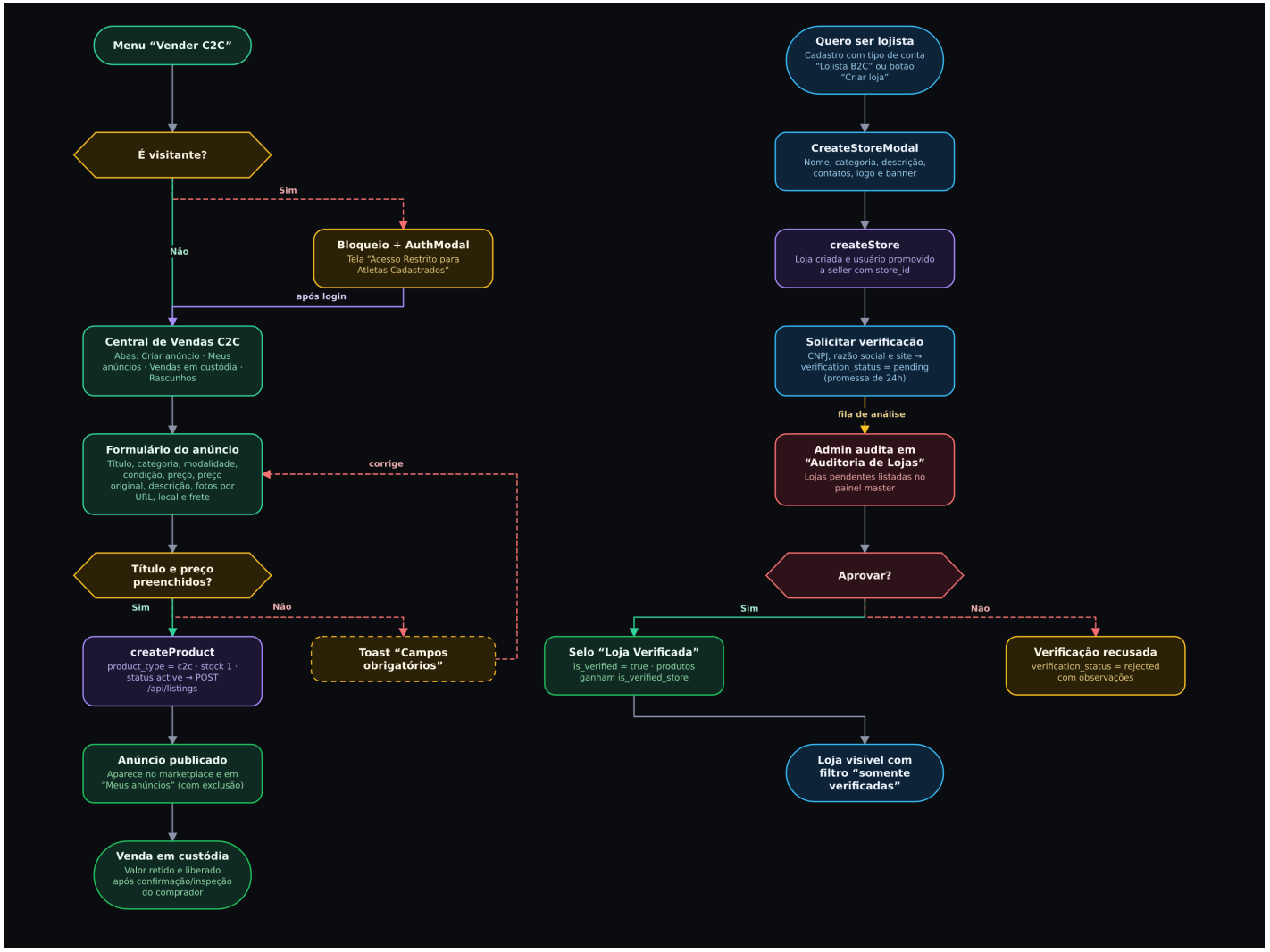
Legenda: Visitante (sem conta) · Atleta / Usuário (role: user) · Alerta / Lacuna / Erro · Confirmação / Sucesso · Sistema / Dados · Super Admin (e-mails master)



4. Jornada de venda - C2C, loja oficial e selo verificado

Dois caminhos de oferta: atleta anunciando equipamento seminovo (C2C com custódia) e lojista abrindo/enviando loja para auditoria de CNPJ, com a homologação feita pelo admin.

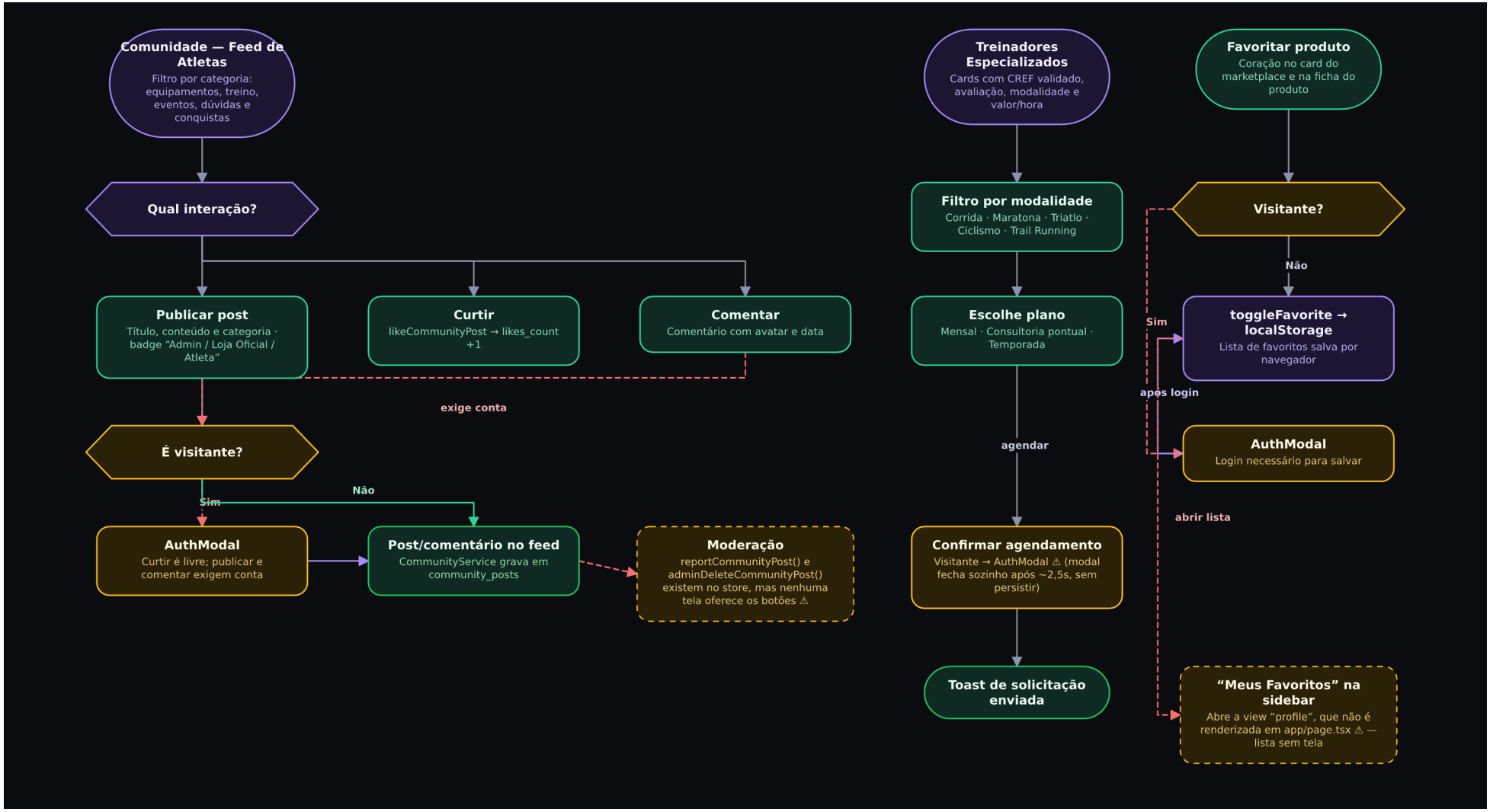
Legenda: Visitante (sem conta) · Atleta / Usuário (role: user) · Lojista (role: seller) · Super Admin (e-mails master) · Alerta / Lacuna / Erro · Confirmação / Sucesso



5. Engajamento - comunidade, treinadores e favoritos

Fluxos sociais e de serviço: feed com 5 categorias, publicação, curtidas e comentários; agendamento de assessoria esportiva com CREF; e a lista de desejos.

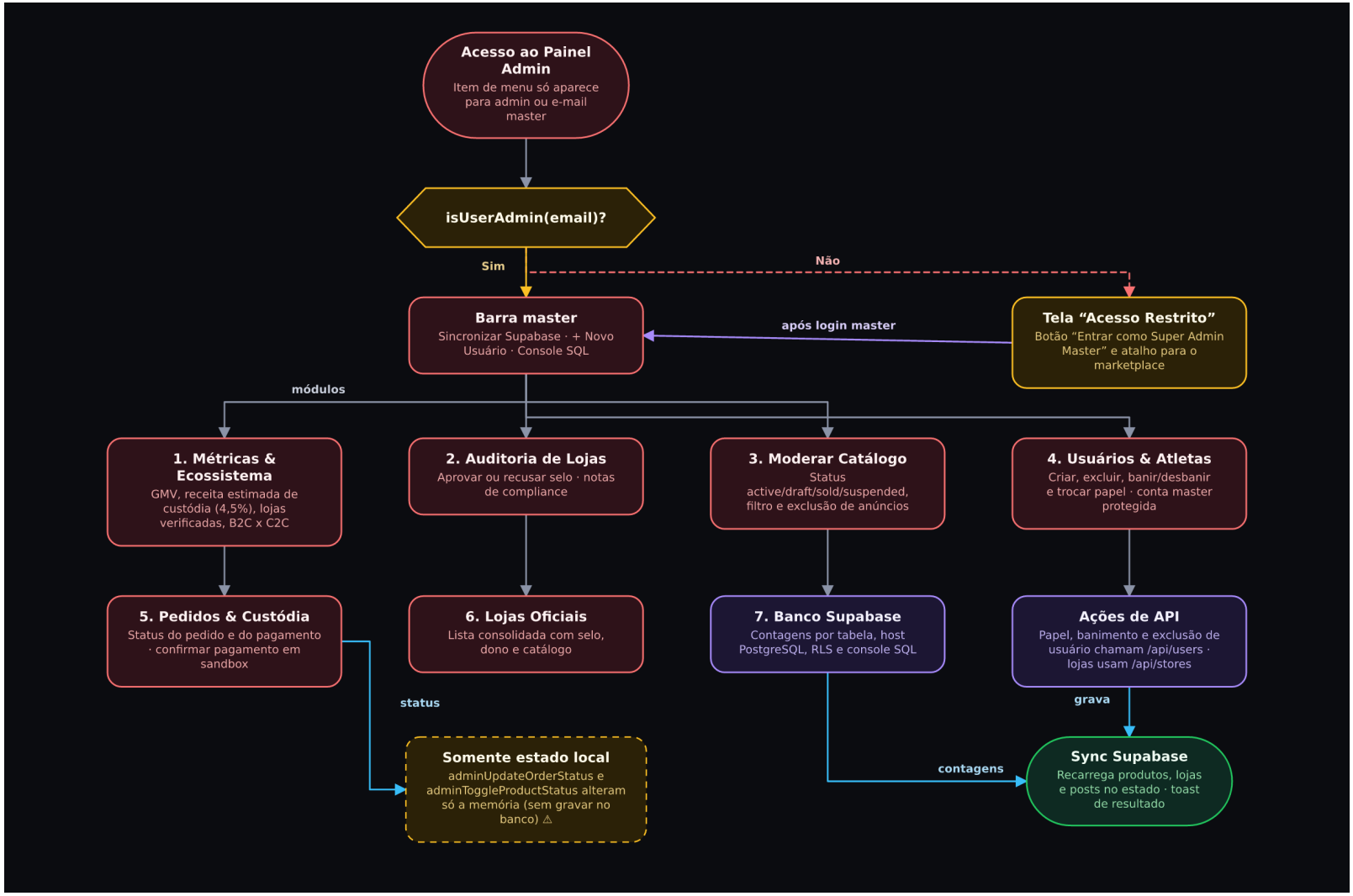
Legenda: Visitante (sem conta) · Atleta / Usuário (role: user) · Sistema / Dados · Alerta / Lacuna / Erro · Confirmação / Sucesso



6. Painel administrativo - 8 módulos executivos

Guarda de acesso por e-mail master, barra de ações rápidas e os módulos: métricas, auditoria de lojas, catálogo, usuários, pedidos/custódia, lojas e banco de dados.

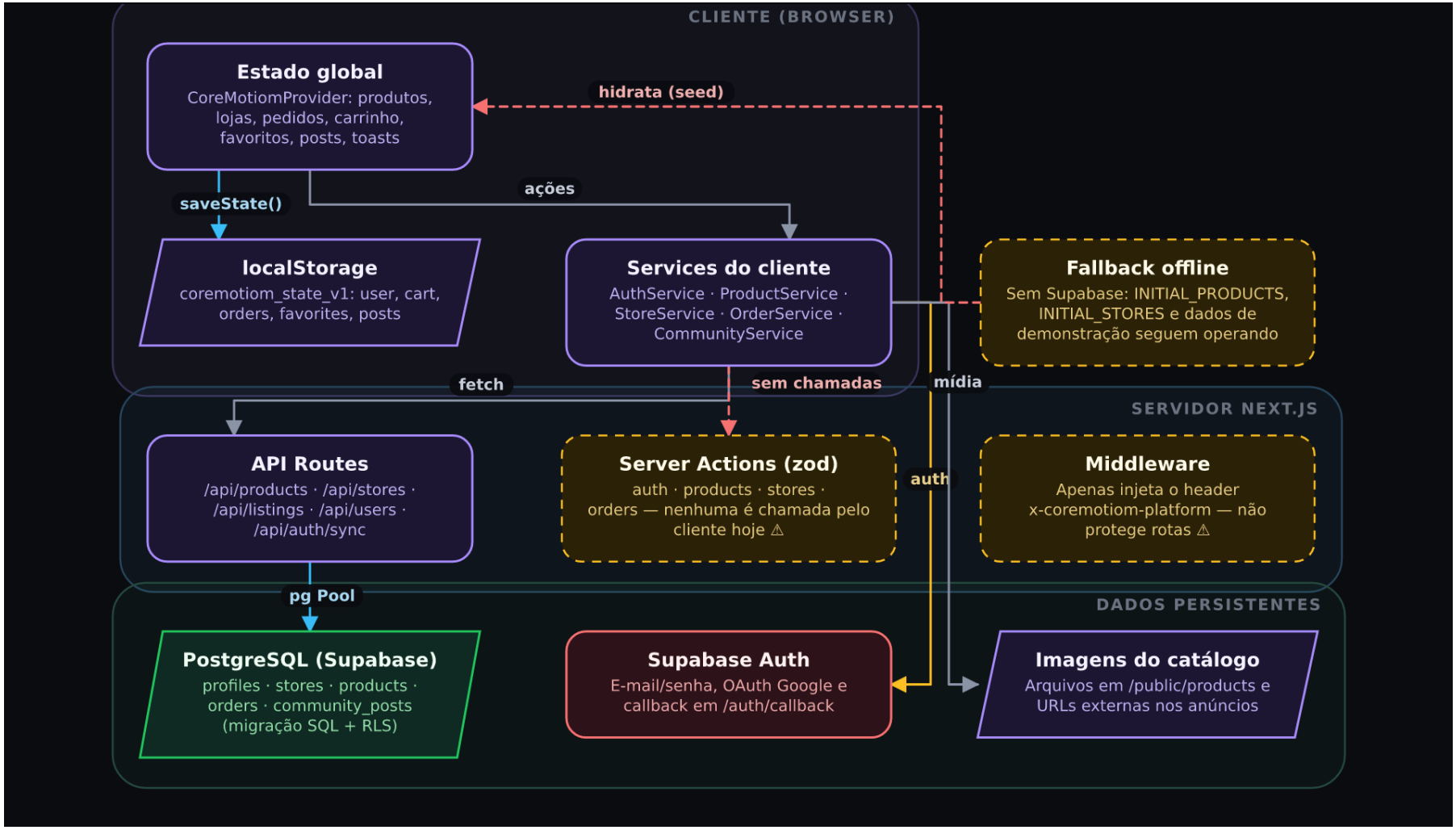
Legenda: Super Admin (e-mails master) · Alerta / Lacuna / Erro · Sistema / Dados · Confirmação / Sucesso



7. Dados, serviços e persistência - onde cada ação é gravada

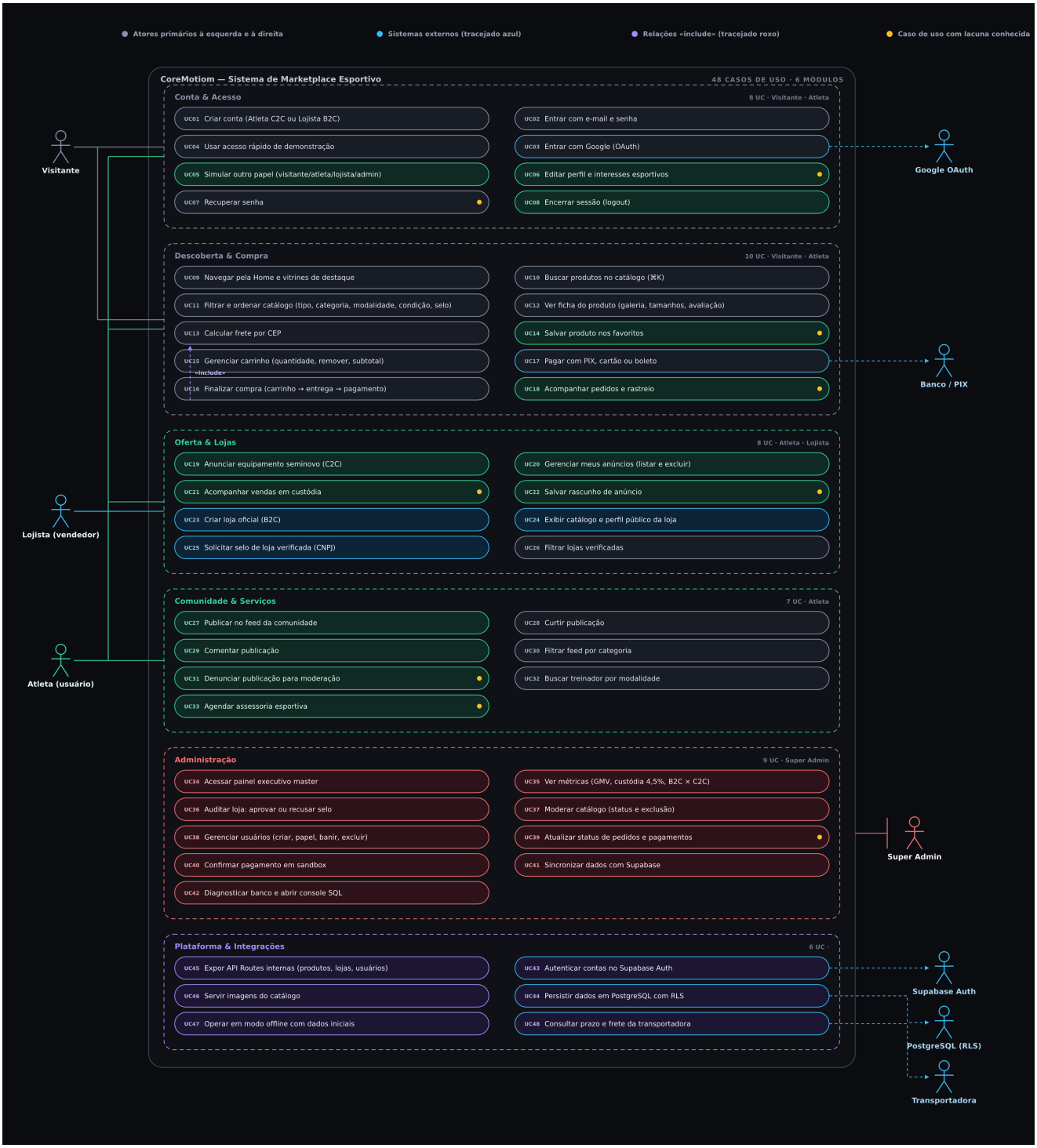
Camadas do sistema: estado global no cliente, services que falam com o Supabase e com as API Routes, pool PostgreSQL com RLS, server actions validadas por zod (hoje sem uso) e o fallback offline.

Legenda: Sistema / Dados · Alerta / Lacuna / Erro · Confirmação / Sucesso · Super Admin (e-mails master)



8. Diagrama de casos de uso

Visão UML dos 48 casos de uso implementados, agrupados em 6 módulos funcionais, com atores primários (visitante, atleta, lojista, admin) e sistemas externos (Google OAuth, Supabase, PostgreSQL, banco/PIX e transportadora). Os identificadores UC correspondem ao catálogo completo logo abaixo.



Atores primários ocupam os corredores laterais (visitante, atleta e lojista à esquerda; super admin à direita). Sistemas externos aparecem em azul tracejado, as relações «include» em roxo e os pontos âmbar marcam casos com lacuna conhecida.

9. Catálogo completo de casos de uso

Casos de uso: 48 | Módulos: 6 | Atores: 4 | Sistemas externos: 5 | Com lacuna: 9

ID	Caso de uso	Atores	Onde está no código
Conta & Acesso			
UC01	Criar conta (Atleta C2C ou Lojista B2C)	Visitante	components/AuthModal.tsx
UC02	Entrar com e-mail e senha	Visitante	services/authService.ts
UC03	Entrar com Google (OAuth)	Visitante + google	app/auth/callback/page.tsx
UC04	Usar acesso rápido de demonstração	Visitante, Atleta	lib/store.tsx · switchRole()
UC05	Simular outro papel (visitante/atleta/lojista/admin)	Atleta	components/Sidebar.tsx
UC06	Editar perfil e interesses esportivos (!) sem tela	Atleta	lib/store.tsx · updateUserProfile()
UC07	Recuperar senha (!) inoperante	Visitante	components/AuthModal.tsx · modo forgot
UC08	Encerrar sessão (logout)	Atleta, Lojista, Super Admin	services/authService.ts · signOut()
Descoberta & Compra			
UC09	Navegar pela Home e vitrines de destaque	Visitante	components/HomeHeroAndHighlights.tsx
UC10	Buscar produtos no catálogo (CmdK)	Visitante	components/Sidebar.tsx
UC11	Filtrar e ordenar catálogo (tipo, categoria, modalidade, condição, selo)	Visitante	components/MarketplaceView.tsx
UC12	Ver ficha do produto (galeria, tamanhos, avaliação)	Visitante	components/ProductDetailModal.tsx
UC13	Calcular frete por CEP	Visitante	components/ProductDetailModal.tsx
UC14	Salvar produto nos favoritos (!) sem tela	Atleta	lib/store.tsx · toggleFavorite()
UC15	Gerenciar carrinho (quantidade, remover, subtotal)	Visitante, Atleta	components/CheckoutModal.tsx
UC16	Finalizar compra (carrinho -> entrega -> pagamento)	Visitante, Atleta	components/CheckoutModal.tsx
UC17	Pagar com PIX, cartão ou boleto	Visitante, Atleta + pix	lib/store.tsx · createOrder()
UC18	Acompanhar pedidos e rastreio (!) sem tela	Atleta	lib/store.tsx · orders
Oferta & Lojas			
UC19	Anunciar equipamento seminovo (C2C)	Atleta	components/SellC2CView.tsx
UC20	Gerenciar meus anúncios (listar e excluir)	Atleta	components/SellC2CView.tsx
UC21	Acompanhar vendas em custódia (!) placeholder	Atleta	components/SellC2CView.tsx
UC22	Salvar rascunho de anúncio (!) placeholder	Atleta	components/SellC2CView.tsx

ID	Caso de uso	Atores	Onde está no código
UC23	Criar loja oficial (B2C)	Lojista	components/CreateStoreModal.tsx
UC24	Exibir catálogo e perfil público da loja	Lojista, Visitante	components/StoresView.tsx
UC25	Solicitar selo de loja verificada (CNPJ)	Lojista	components/StoresView.tsx
UC26	Filtrar lojas verificadas	Visitante	components/StoresView.tsx
Comunidade & Serviços			
UC27	Publicar no feed da comunidade	Atleta	components/CommunityView.tsx
UC28	Curtir publicação	Visitante	services/communityService.ts
UC29	Comentar publicação	Atleta	components/CommunityView.tsx
UC30	Filtrar feed por categoria	Visitante	components/CommunityView.tsx
UC31	Denunciar publicação para moderação (!) sem UI	Atleta	lib/store.tsx · reportCommunityPost()
UC32	Buscar treinador por modalidade	Visitante	components/CoachesView.tsx
UC33	Agendar assessoria esportiva (!) não persiste	Atleta	components/CoachesView.tsx
Administração			
UC34	Acessar painel executivo master	Super Admin	components/AdminDashboard.tsx
UC35	Ver métricas (GMV, custódia 4,5%, B2C × C2C)	Super Admin	components/AdminDashboard.tsx
UC36	Auditar loja: aprovar ou recusar selo	Super Admin	lib/store.tsx · adminVerifyStore()
UC37	Moderar catálogo (status e exclusão)	Super Admin	lib/store.tsx · adminToggleProductStatus()
UC38	Gerenciar usuários (criar, papel, banir, excluir)	Super Admin	app/api/users/route.ts
UC39	Atualizar status de pedidos e pagamentos (!) só memória	Super Admin	lib/store.tsx · adminUpdateOrderStatus()
UC40	Confirmar pagamento em sandbox	Super Admin	services/orderService.ts
UC41	Sincronizar dados com Supabase	Super Admin	lib/store.tsx · adminSyncSupabase()
UC42	Diagnosticar banco e abrir console SQL	Super Admin	components/SupabaseConfigModal.tsx
Plataforma & Integrações			
UC43	Autenticar contas no Supabase Auth	+ supabaseAuth	services/supabaseClient.ts
UC44	Persistir dados em PostgreSQL com RLS	+ postgres	lib/db-queries.ts
UC45	Expor API Routes internas (produtos, lojas, usuários)		app/api/*
UC46	Servir imagens do catálogo		public/products/*
UC47	Operar em modo offline com dados iniciais		lib/initial-data.ts
UC48	Consultar prazo e frete da transportadora	+ correios	components/ProductDetailModal.tsx

10. Papéis e permissões por tela

Como o app decide o que cada perfil vê e faz (lib/store.tsx + guardas em cada componente).

Capacidade	Visitante	Atleta (user)	Lojista (seller)	Admin master
Navegar Home, Marketplace, Lojas, Treinadores e Feed	sim	sim	sim	sim
Busca global (CmdK) e filtros do marketplace	sim	sim	sim	sim
Curtir post da comunidade	sim	sim	sim	sim
Favoritar produto	não -> AuthModal	sim	sim	sim
Adicionar ao carrinho e finalizar compra	sim (pedido guest)	sim	sim	sim
Publicar post e comentar	não -> AuthModal	sim	sim	sim
Agendar treinador	não -> AuthModal	sim	sim	sim
Anunciar equipamento C2C (SellC2CView)	não -> AuthModal	sim	sim	sim
Criar loja oficial (CreateStoreModal)	não	sim (vira seller)	sim	sim
Solicitar selo de loja verificada	não	não	sim (dono)	sim
Ver o item "Painel Admin" na sidebar	não	não	não	sim
Aprovar lojas, moderar catálogo e usuários	não	não	não	sim
Acesso rápido de demonstração (switchRole)	sim	sim	sim	sim

11. Máquina de estados do domínio

Todos os estados previstos nos tipos do projeto e quem os movimenta.

Entidade	Status possíveis	Quem altera
Order.order_status	pending_payment -> paid -> preparing -> shipped -> delivered · cancelled	Checkout (criação) e Admin (confirmar pagamento / alterar status)
Order.payment_status	pending · completed · failed · refunded	Checkout e Admin (sandbox)
Store.verification_status	none -> pending -> verified / rejected	Lojista solicita; Admin homologa
Product.status	active · draft · sold · suspended	Lojista (criação) e Admin (moderação)
UserProfile.role	visitor · user · seller · admin	Cadastro e Admin (updateUserRole)
UserProfile.is_banned	false -> true (ban_reason)	Admin (banir / desbanir)

12. Lacunas e riscos encontrados no fluxo

Pontos em que o caminho do usuário termina em tela vazia, em ação sem persistência ou em risco operacional.

View "profile" não existe · Navegação

A sidebar leva para activeView = "profile" em "Meus Favoritos" e no avatar mobile, mas app/page.tsx não renderiza esse bloco - a tela fica vazia, sem lista de favoritos, dados do perfil ou pedidos do usuário.

View "news" declarada e sem tela · Navegação

ActiveView inclui "news" e o store carrega INITIAL_NEWS, mas nenhum componente consome essas notícias.

SmartScanView órfão · Funcionalidade

components/SmartScanView.tsx (análise de desgaste/pace com IA) tem 356 linhas e não é importado por nenhuma view do app.

Componentes duplicados · Manutenção

AuthModal, MarketplaceView, ProductCard, ProductDetailModal, StoresView, CreateStoreModal, Footer e AdminDashboard existem na raiz de components/ e também em subpastas que apenas reexportam - risco real de editar o arquivo errado.

Checkout não coleta bairro/número · Checkout

O formulário pede apenas endereço, cidade e UF, mas grava "Bela Vista" e número "1000" fixos; o shippingMethod é sempre "pac". O checkoutSchema do zod exige neighborhood e number - o server action de checkout rejeitaria esses dados.

Server actions sem uso · Arquitetura

serverLogin, serverSignUp, serverProcessCheckout, serverCreateProduct e serverCreateStore nunca são chamadas: os componentes falam direto com Supabase/services. Duas trilhas concorrentes de escrita, com validações diferentes.

Visitante finaliza compra · Regra de negócio

O carrinho e o CheckoutModal não exigem login; createOrder usa user?.id || "guest-user". Pedidos de visitante não têm dono para consulta posterior.

Moderação de posts inacessível · Comunidade

reportCommunityPost e adminDeleteCommunityPost existem no store, mas nenhum componente expõe botão de denúncia ou exclusão - os campos is_reported/report_reason nunca são acionados pela UI.

Recuperação de senha inoperante · Autenticação

O modal de "Esqueci a senha" só exibe uma mensagem informativa; AuthService.resetPassword existe e não é chamado, e não há rota /auth/reset-password.

Status de pedido e produto só em memória · Dados

adminUpdateOrderStatus e adminToggleProductStatus alteram apenas o estado do React. Não há função de ordens em lib/db-queries.ts - pedidos vivem em localStorage/Supabase, sem trilha administrativa persistida.

Credenciais no código · Segurança

lib/db.ts traz DEFAULT_DATABASE_URL com usuário e senha do PostgreSQL em texto puro como fallback. Essas credenciais devem sair do repositório e virar variável de ambiente (com rotação).

Middleware não protege rotas · Segurança

middleware.ts apenas adiciona o header de plataforma; /admin e APIs internas não têm verificação de sessão no servidor - a regra de admin é só client-side (isUserAdmin por e-mail).

Favoritos, carrinho e pedidos sem tabela · Dados

A migração SQL cobre profiles, stores, products, orders e community_posts; carrinho e favoritos existem apenas no localStorage e não sincronizam entre dispositivos.

Agendamento de treinador não persiste · Funcionalidade

handleConfirmBooking só dispara toast e fecha o modal após 2,5s: não há tabela, serviço ou registro de agendamento.

13. Arquivos-chave por fluxo

Onde mexer para evoluir cada jornada.

Fluxo	Ponto de entrada	Regras e dados
Shell e navegação	app/layout.tsx · app/page.tsx · components/Sidebar.tsx	lib/store.tsx (ActiveView, modais globais, toasts)
Autenticação	components/AuthModal.tsx · app/auth/callback/page.tsx	services/authService.ts · app/api/auth/sync/route.ts · lib/db-queries.ts
Compra	components/HomeHeroAndHighlights.tsx · MarketplaceView · ProductDetailModal · CheckoutModal	services/orderService.ts · lib/schemas/order.schema.ts
Venda C2C	components/SellC2CView.tsx · ProductCard.tsx	services/productService.ts · app/api/listings/route.ts · lib/schemas/product.schema.ts
Lojas e selo	components/StoresView.tsx · CreateStoreModal.tsx · AdminDashboard.tsx	services/storeService.ts · app/api/stores/route.ts · lib/schemas/store.schema.ts
Comunidade	components/CommunityView.tsx	services/communityService.ts · lib/initial-data.ts
Treinadores	components/CoachesView.tsx	INITIAL_COACHES (lib/initial-data.ts)

Fluxo	Ponto de entrada	Regras e dados
Administração	components/AdminDashboard.tsx	lib/store.tsx (ações admin) · app/api/users/route.ts · app/api/products/route.ts
Banco e infraestrutura	lib/db.ts · lib/db-queries.ts · middleware.ts	supabase/migrations/20250101000000_coremotiom_initial_s

14. Manutenção deste documento

O PDF é gerado por script a partir do mesmo núcleo que produz o HTML - não edite os arquivos à mão. Ao mudar uma jornada, atualize scripts/lib/flowchart-core.mjs (fluxos) ou scripts/lib/usecase-core.mjs (casos de uso) e rode os geradores.

- node scripts/generate-user-flowchart.mjs ..... regenera docs/fluxograma-usuario.html
- node scripts/generate-user-flowchart-pdf.mjs ..... regenera docs/fluxograma-usuario.pdf
- npm i -D @resvg/resvg-js ..... rasterizador usado para embutir os diagramas